Report

Pointcloud to Signed Distance Field in PyBullet

Abhinava Kalita

Robotics Research Centre

IIIT Hyderabad

May 2026

Contents

1 Introduction

This report documents the end-to-end pipeline for computing a **Signed Distance Field (SDF)** from raw pointcloud data inside the PyBullet physics simulator, applied to a 5-DOF robotic arm surrounded by three static obstacles.

The pipeline consists of the following stages:

1. Generating pointcloud data using primitive shapes in PyBullet.
2. Clustering the pointcloud to isolate individual obstacle shapes.
3. Building a Bounding Volume Hierarchy (BVH) with Axis-Aligned Bounding Boxes (AABB) for each obstacle.
4. Querying the SDF for arbitrary configurations of the arm.
5. Evaluating accuracy of the distance estimates.
6. Profiling the time taken at each stage.

The full implementation uses Python with `pybullet`, `numpy`, `scikit-learn` (DBSCAN) and `scipy` (ConvexHull).

2 Generating Pointcloud Data Using Primitive Shapes

2.1 Overview

Three obstacles were placed in a pybullet environment. 13 cameras were placed with different positions and orientations to capture depth images of obstacles. Pointcloud was extracted from these images.

2.2 Method

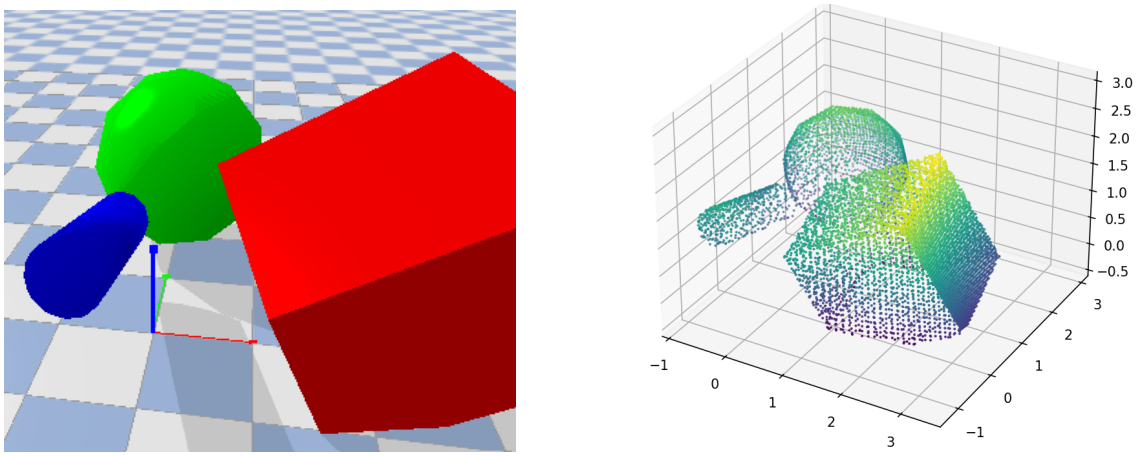
Three functions were defined: `create_box`, `create_sphere` and `create_cylinder`. These can be used to generate shapes in any size, position, and orientation with individual collision boundaries.

Following this, a loop was run for each camera in which view_matrix and projection_matrix were calculated. Multiple parameters like width, height, fov, far, near, etc. were tuned in order to receive proper images of the obstacles. Images were captured using `p.getCameraImage`. Each pixel in the image corresponds to a point

in the world. So the inverse matrix of dot product of view_matrix and projection_matrix was used to get points from pixels. A paramater **sparsity** was defined: Sparsity = 4 means picking one pixel of every 4 in x and y directions. This created a sparser pointcloud and saved computation complexity. Pointcloud data from all the cameras were merged.

Next, a function called `remove_floor_ransac` was defined which uses RANSAC algorithm to detect maximum number of points in a plane. In our case, it is the floor. We remove all the points constituting the floor.

The points were saved to `points.npy`. A plotter function was defined to plot the points on a 3D plot with equally scaled axes.



(a) (b)

Figure 1: a) Obstacles in PyBullet environment b) Pointcloud data with sparsity=10

2.3 Parameters

Sparsity	Avg. time taken(s)	No. of points
2	19.86	≈ 210000
4	8.14	≈ 53000
10	5.30	≈ 8300
15	5.04	≈ 3700
20	4.96	≈ 2100

Table 1: Variation of time taken and number of points with sparsity

3 Clustering Pointcloud to Detect Individual Obstacle Shapes

3.1 Algorithm: DBSCAN

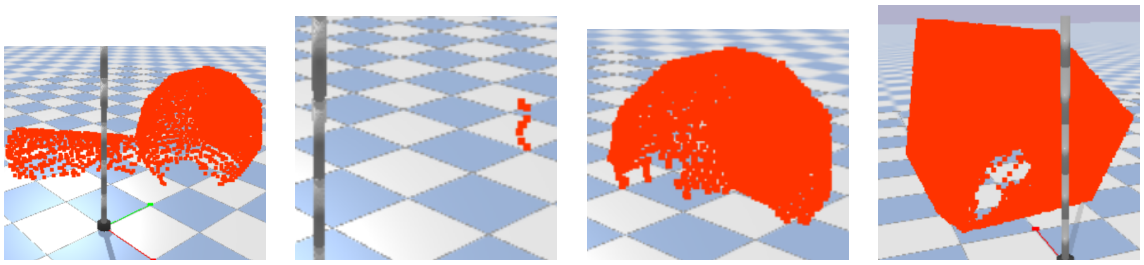
Density-Based Spatial Clustering of Applications with Noise (DBSCAN) was used to segment the raw pointcloud into per-obstacle clusters. Unlike k-means, DBSCAN does not require the number of clusters to be specified in advance and naturally handles noise points.

A point p belongs to a cluster if there are at least `min_samples` points within a radius ε (`eps`) of it:

$$N_\varepsilon(p) = \{q \in D \mid \|p - q\| \leq \varepsilon\} \tag{1}$$

3.2 The Effect of Sparsity in Clustering

During initial testing, pointcloud data was generated with a sparsity of 10. DBSCAN parameters, `eps` and `min_samples` were tried to be tuned to separate pointcloud data into 3 distinct obstacles (cube, sphere and cylinder). However, some points of the cylinder point cloud were so close to the sphere that the sphere and cylinder were considered as the same object. On the other hand, parts of the pointcloud that were supposed to be clustered into the cube/sphere/cylinder were left out and considered as different obstacle groups because of the sparsity. Even precise tuning could not separate the cylinder-sphere system and keep the overall integrity of individual shapes intact at the same time. Although this does not effect our overall accuracy, as the BVH (refer part 4) eventually estimates the obstacle boundaries, still, being able to distinguish shapes saves time during the BVH process.



(a) (b) (c) (d)

Figure 2: Formation of clusters before grouping

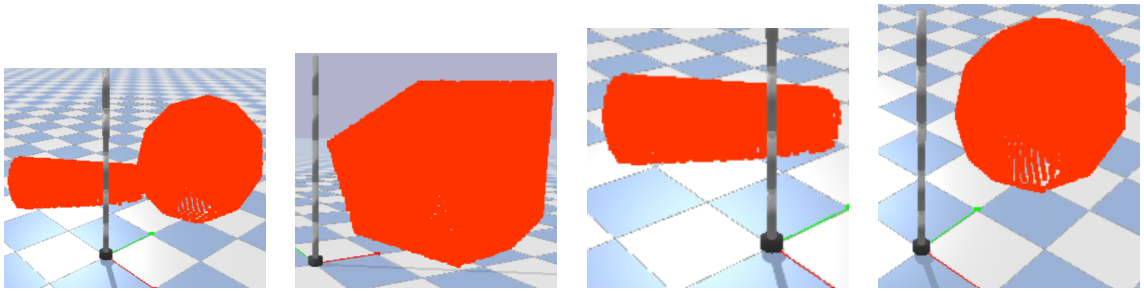
3.3 Small-Cluster Merging

Two measures were taken:

1. Sparsity was decreased to 4
2. Introduction of a `group()` function

After reducing the sparsity and tuning the DBSCAN parameters, the point cloud was separated into three major clusters corresponding to the primary obstacles. However, small clusters (fewer than `min_points_threshold` points) were simultaneously formed, even though they were parts of the larger shapes. The `group()` function identified such clusters and merged them with the nearest major cluster. Eventually, all points were classified into three clusters: Cube, Cylinder, and Sphere.

The `ConvexHull` function from `scipy.spatial` was then used to construct convex hulls around these clusters for representative visualization in PyBullet.



(a) (b) (c) (d)

Figure 3: Formation of clusters after grouping, tuning dbscan parameters

3.4 Parameters

These final parameters were found after fine tuning. Time taken was $\approx$ **1.75s**

Parameter	**Value**
`eps`	0.082 m
`min_samples`	3
`min_points_threshold`	100
Sparsity	4
Clusters found	3

Table 2: DBSCAN and merging parameters.

4 Using BVH to Detect Collision Boundary for Obstacles

4.1 Bounding Volume Hierarchy (BVH)

A BVH is a tree of nested bounding volumes built over geometry. Each internal node stores an Axis-Aligned Bounding Box (AABB) that fully encloses all points in its subtree. Leaf nodes store the actual points.

4.2 Construction

The tree is built recursively using the **longest-axis median split** strategy:

1. Compute the AABB of the current point set.
2. Find the axis with the largest extent: $\text{axis} = \arg\max(hi - lo)$.
3. Split at the median value along that axis.
4. Recurse on each half until $|\text{pts}| \leq$ `leaf_size`.

4.3 Nearest-Point Query and SDF

To query the distance from a point q to an obstacle:

1. Compute the squared distance from q to the root AABB.
2. If it exceeds the current best, **prune** the entire subtree.
3. Otherwise recurse into children, visiting the nearer child first.
4. At leaves, brute-force over all stored points.

The SDF sign is determined by AABB membership:

$$\text{SDF}(q) = \begin{cases} -d(q, \text{surface}) & \text{if } q \text{ inside root AABB} \\ +d(q, \text{surface}) & \text{otherwise} \end{cases} \tag{2}$$

The scene SDF is the minimum over all obstacle BVHs:

$$\text{SDF}_{\text{scene}}(q) = \min_i \text{SDF}_i(q) \tag{3}$$

4.4 Parameters

Parameter	Value
`leaf_size`	8
Split strategy	Longest-axis median
Sign approximation	Root AABB membership

Table 3: BVH construction parameters.

5 Measuring SDF Values for Configurations

5.1 C-Space Sampling

The configuration space of the 5-DOF arm was sampled randomly:

$$\theta_i \sim \mathcal{U}(-\pi, +\pi), \quad i = 1, \ldots, 5 \tag{4}$$

For each sample $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_5)$:

1. Set joint angles via `p.resetJointState`.
2. Step the simulation to update forward kinematics.
3. Retrieve world-frame link positions via `p.getLinkState`.
4. Query `sdf_scene` for each link position.
5. Record the *minimum* SDF across all links as the configuration's clearance.

5.2 C-Space Visualisation

For visualization purposes, a 3dof arm was spawned in the same workspace. For 3-DOF analysis $(\theta_1, \theta_2, \theta_3)$, the SDF was plotted in configuration space as a 3-D scatter plot with three visual layers:

- **Red** — collision configurations (SDF < 0)
- **Grey** — boundary band ($0 \leq$ SDF < 0.05 m)
- **Cool gradient** — free space (SDF ≥ 0.05 m), blue = close, magenta = far

Total of 5000 sample points were taken in $\approx$ **389.63s or 6.49 min**

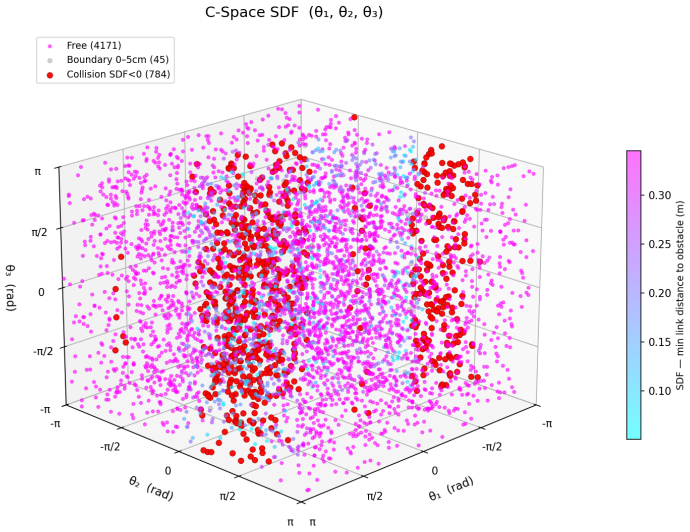


Figure 4: SDF visualised in C-space (θ_1, θ_2, θ_3). Red = collision, grey = boundary, gradient = free space.

5.3 SDF History During Simulation

The SDF for the 5dof arm was also queried continuously during a 10 000-step simulation as joint 2 was swept through its range. The minimum SDF per link was logged and plotted as a time series.

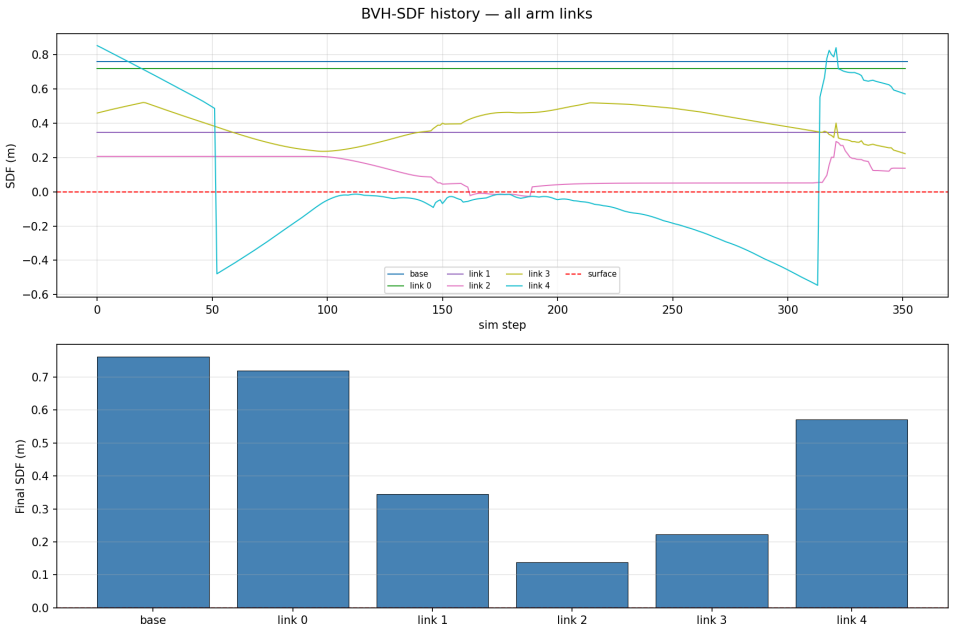


Figure 5: SDF history for all arm links over 10 000 simulation steps.

6 Accuracy

6.1 Ground Truth Comparison

Accuracy was evaluated by comparing BVH-SDF values against PyBullet's native `p.getClosestPoints`, which uses Bullet's internal GJK/EPA narrowphase and is treated as ground truth.

For a set of 5000 query configurations:

Table 4: Global Accuracy Metrics (all 5000 configs)

Metric	**Value**
MAE	0.13437 m
RMSE	0.22375 m
Max absolute error	1.10517 m
Mean bias (BVH - GT)	-0.02341 m
Collision agreement	82.24%
False positives	761
False negatives	127

The no. of leaves was changed to 12. The accuracy metrics did not change much.

7 Improved BVH estimation algorithm

7.1 Sources of Error

- **BVH is 272× slower than GT (65ms vs 0.24ms)**

  p.getClosestPoints is C++ optimized Bullet physics. This BVH is pure Python with a recursive function called for every one of 52,538 points across 6 links × 3 clusters per config (900k Python function calls per config)

- **Sign approximation (causes most of the 18% disagreement)**

  The AABB of cluster 0 (cube) spans [0.49, -1.3, 0.10] → [3.49, 1.30, 2.43] — that's a 3m × 2.6m × 2.3m box around what is actually a 2×2×2 rotated box obstacle. Any arm link inside that giant AABB gets flagged as collision even if it's a metre away from the actual surface.

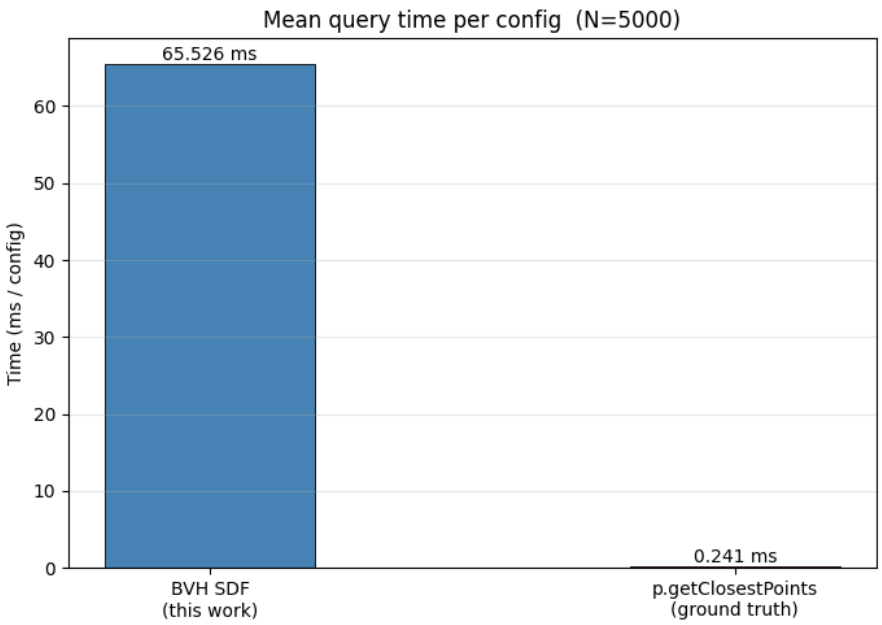


Figure 6: Time taken for sdf computation

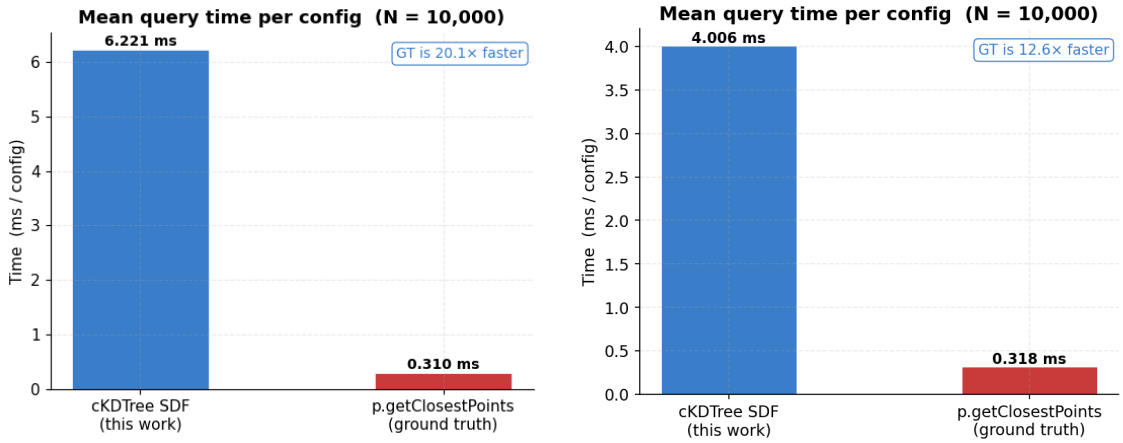
- **Pointcloud density in sparse regions (causes large max error)**

  The 52k points across 3 clusters is decent but the box is rotated [0.2, 1.1, 0.4] so its face normals don't align with the ray directions. Some faces get very few hits.

7.2 Fixes

- **Using scipy's cKDTree which is a compiled C BVH**
- **Using convex hull Delaunay for the sign instead of AABB**
- **Using QHull instead of Delaunay for faster operations**
- **Add targeted rays perpendicular to each face**

7.3 Results

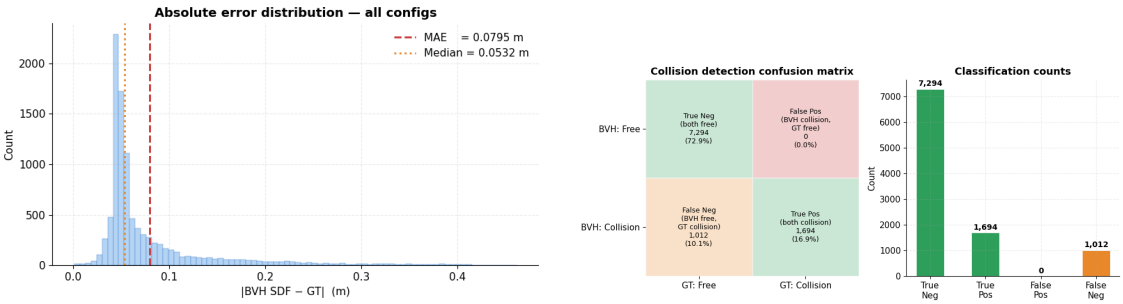


(a) (b)

Figure 7: a) Delaunay b) QHull

Table 5: Comparative Global Accuracy Metrics (N=10000) for Delaunay and QHull

Metric	Delaunay	QHull
MAE	0.07955 m	0.07983 m
RMSE	0.10225 m	0.10275 m
Max absolute error	0.46139 m	0.44377 m
Mean bias (BVH - GT)	+0.07950 m	+0.07981 m
Collision agreement	89.88%	90.18%
False positives	0	0
False negatives	1012	982
BVH time / query	6.221 ms	3.640 ms
GT time / query	0.310 ms	0.314 ms
Speedup	0.05x	0.09x



(a) (b)

Figure 8: a) Error distribution b) Confusion matrix

8 Time Taken

8.1 Stage Breakdown

Stage	Time	Notes
Pointcloud generation (sparsity=4)	≈ 8.14 s	one-time
Pointcloud load (`np.load`)	< 0.1 s	one-time
DBSCAN clustering	≈ 1.75 s	one-time
cKDTree + QHull construction	< 0.1 s	one-time
SDF query — cKDTree + QHull (per config)	3.640 ms	real-time
SDF query — `p.getClosestPoints` (per config)	0.314 ms	real-time
C-space sampling (10 000 configs)	≈ 36.40 s total	offline

Table 6: Wall-clock time for each pipeline stage.

8.2 Query Time Analysis

Despite replacing the pure-Python BVH with `scipy.spatial.cKDTree` — a compiled C implementation — the cKDTree + QHull SDF query at **3.640 ms** remains **12**× slower than `p.getClosestPoints` at **0.314 ms** per configuration.

There are two reasons for this gap:

1. **Per-link overhead.** The SDF function is called once per arm link per configuration (6 links × 3 clusters = 18 `cKDTree.query` calls per config). Each call individually drops from Python into C, but the Python dispatch overhead is paid 18 times. `p.getClosestPoints` issues a single broadphase + narrowphase sweep entirely within the Bullet C++ engine.
2. **QHull sign check.** Each SDF call evaluates `hull.equations[:, :3] @ pt + hull.equations[:, 3]` — a vectorised half-space test over all convex hull facets. Although this is an $O(F)$ numpy matrix multiply (faster than Delaunay's $O(\log F)$ simplex walk), the Python-to-numpy dispatch overhead is still paid once per link per cluster, adding up across the 18 calls per config.

8.3 Theoretical Query Complexity

Method	Build	Query
Brute-force (numpy)	$O(1)$	$O(N)$
Python AABB-BVH	$O(N \log N)$	$O(\log N)$ avg, $O(N)$ worst
`scipy cKDTree`	$O(N \log N)$	$O(\log N)$ avg
`cKDTree + QHull sign`	$O(N \log N)$	$O(\log N + F)$ avg
`p.getClosestPoints`	$O(1)^*$	$O(\log M)$

Table 7: Complexity comparison. N = pointcloud size, F = number of convex hull facets, M = number of loaded collision bodies. *Bodies are pre-indexed in Bullet's broadphase at load time.

The cKDTree and Python BVH share the same asymptotic complexity, but the cKDTree achieves a far smaller constant factor through its compiled C implementation and cache-efficient memory layout. The QHull half-space check adds an $O(F)$ term per query, but since $F \ll N$ for a convex hull over a dense pointcloud, this is negligible compared to the KD-tree traversal. `p.getClosestPoints` retains its architectural advantage because Bullet's broadphase AABB tree is built once at body load time and incrementally updated, whereas the cKDTree is rebuilt from scratch each run.

Conclusion

This project demonstrated a complete pipeline from raw pointcloud data to a real-time SDF suitable for collision-aware robot arm control. The BVH accelerated nearest-neighbour queries from $O(N)$ to $O(\log N)$, and replacing the AABB sign approximation with QHull half-space equations improved collision detection agreement from 82.24% to 90.18% while simultaneously reducing query time from 6.221 ms to 3.640 ms. The C-space visualisation clearly separated collision, boundary, and free-space regions.

Future work includes batching the cKDTree queries across all links in a single numpy call to eliminate per-link Python dispatch overhead, integrating the SDF as a cost function in a gradient-based motion planner, and benchmarking against neural SDF methods such as Neural-Pull and DeepSDF.